QUANTUM SECURITY INTELLIGENCE

# Security & Post-Quantum Readiness Audit Report

secp256k1 @ 1b4181f93f7a · Bitcoin Core (libsecp256k1) — public repository (passive review)

| | |
|---|---|
| Organization | QSI |
| Report date | 2026-10-29 05:36 UTC |
| Report version | 1 |
| Audit run | 978b0176-06de-45cc-9c2d-8afbe12829e1 · Document audit · 2026-10-29 05:36 UTC |
| Analyzer engine | qsi-engine 1.2.0 |
| Risk profile | Crypto exchange, custodian or wallet |
| Content SHA-256 | `169d88e86f2f6d7d0b7b36eaadbd0524a11b958affb0f07abddec70ff1d2820a` |
| Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json) | `5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e` |

# Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

# Executive summary

| 5 | 0 | 0 | 3 | 0 |
|---|---|---|---|---|
| Findings in total | Critical | High | Medium | Low |

| Classification | Count |
|---|---|
| Confirmed technical vulnerability | 0 |
| Known vulnerable component | 0 |
| Confirmed configuration error | 0 |
| Potential cryptographic risk | 0 |
| Long-term post-quantum risk | 5 |
| Improvement recommendation | 0 |
| Insufficient data | 0 |

## Top 5

| ID | Severity | Detailed findings | Priority score |
|---|---|---|---|
| QSI-2026-06409 | Medium | Quantum-vulnerable ECDSA: transaction and message signing (10 file(s)) confirmed by the analyst | 41.8 |
| QSI-2026-06410 | Medium | Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (3 file(s)) confirmed by the analyst | 41.8 |
| QSI-2026-06411 | Medium | Quantum-vulnerable ECDH: role not determined (1 file(s)) confirmed by the analyst | 40.2 |
| QSI-2026-06408 | Info | Quantum-vulnerable ECDSA: role not determined (16 file(s)) confirmed by the analyst | 31.2 |
| QSI-2026-06412 | Info | Quantum-vulnerable Schnorr (BIP-340): role not determined (1 file(s)) confirmed by the analyst | 31.2 |

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Analyst review: 5 confirmed, 0 not applicable, 0 dismissed as false positives.

**Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography.** Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

# Scope and boundaries

**Mode:** Document audit

**Repository:** https://github.com/bitcoin-core/secp256k1 @ 1b4181f93f7a53639223ced4c347d0d0d2c2ff9c — evidence links open the lines at this commit.

## Supplied inputs

99 files, 4185409 bytes in total, 0 redaction(s); the per-file SHA-256 list is in appendix C.

# Methodology

The audit was performed by the QSI engine 1.2.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

## Coverage of checks

| Area | Status | Count | Examples |
|---|---|---|---|
| source_code | checked | 99 | contrib/lax_der_parsing.c: 148 lines scanned with 16 rule patterns and 123 registry patterns; contrib/lax_der_parsing.h: 97 lines scanned with 16 rule patterns and 123 registry patterns; contrib/lax_der_privatekey_parsing.c: 112 lines scanned with 16 rule patterns and 123 registry patterns |

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

# Assets examined and not examined

**Supplied inputs:** 99 (input files: C)

# Architecture overview

Static review of the source tree secp256k1 at 1b4181f93f7a53639223ced4c347d0d0d2c2ff9c. 99 files analysed; of 189 files, 37 are test, fuzzing, benchmark or example code and were left out, 53 are not read by any analyzer, 0 exceed the per-file limit and 0 did not fit the total limit. No requests were sent to the owner's systems.

# Overall security posture

**100**
Indicative posture index (0–100)

**100.0%**
Quantum-vulnerable share of inventory

**38**
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

# Findings by severity




| ID | Detailed findings | Classification | Severity | Confidence |
|---|---|---|---|---|
| QSI-2026-06409 | Quantum-vulnerable ECDSA: transaction and message signing (10 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-06410 | Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (3 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-06411 | Quantum-vulnerable ECDH: role not determined (1 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-06408 | Quantum-vulnerable ECDSA: role not determined (16 file(s)) | Long-term post-quantum risk | Info | Medium |
| QSI-2026-06412 | Quantum-vulnerable Schnorr (BIP-340): role not determined (1 file(s)) | Long-term post-quantum risk | Info | Medium |

# Detailed findings

QSI-2026-06409 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable ECDSA: transaction and message signing (10 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** The row that matters more than any other in the portfolio. This is the implementation every audited self-custody wallet reaches in the end: Jade vendors libwally, which vendors this; Krux pins embit, which nests a zkp fork of this; SeedSigner pins embit from PyPI, which binds to this through ctypes; rust-bitcoin reaches it through rust-secp256k1's secp256k1-sys. The anchors are the public surface rather than internals — include/secp256k1.h:75, src/secp256k1.c:20 and :373, src/ecdsa.h and src/ecdsa_impl.h, and the recovery module at src/modules/recovery/main_impl.h:74. Exposure is forward-only for a key never used, and retroactive in the practical sense for any key whose public form is already on chain, which is most of them. A post-quantum signature standard does replace this, by consensus change rather than by a library release — so the library is not where the fix happens, which is exactly why knowing its shape matters.

51 reference(s) to ECDSA across 10 file(s), e.g. include/secp256k1.h:75. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

| | |
|---|---|
| Affected asset | source-code:ECDSA:transaction_signing |
| Data source | include/secp256k1.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.2.0 · 2026-10-29 05:36 UTC |
| Priority score | 41.8 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include/secp256k1.h:75 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · include/secp256k1.h · line 75 · SHA-256 b6351800fde3e84a... · File f9af5be29b0fbe1a...

```
74 |  */
75 | typedef struct secp256k1_ecdsa_signature {
76 |     unsigned char data[64];
```

file_excerpt · include/secp256k1.h · line 77 · SHA-256 05e9164b839b3186... · File f9af5be29b0fbe1a...

```
 76 |     unsigned char data[64];
 77 | } secp256k1_ecdsa_signature;
 78 |
```

file_excerpt · include/secp256k1.h · line 541 · SHA-256 d969cc0b74d42823... · File f9af5be29b0fbe1a...

```
540 |  */
541 | SECP256K1_API int secp256k1_ecdsa_signature_parse_compact(
542 |     const secp256k1_context *ctx,
```

file_excerpt · include/secp256k1_ecdh.h · line 4 · SHA-256 1b629d577e7d70d5... · File ad314d14cb79a79d...

```
 3 |
 4 | #include "secp256k1.h"
 5 |
```

file_excerpt · include/secp256k1_ellswift.h · line 4 · SHA-256 1b629d577e7d70d5... · File 620db468b1c13968...

```
 3 |
 4 | #include "secp256k1.h"
 5 |
```

file_excerpt · include/secp256k1_extrakeys.h · line 4 · SHA-256 1b629d577e7d70d5... · File 7c3468c9fd192ad1...

```
 3 |
 4 | #include "secp256k1.h"
 5 |
```

file_excerpt · include/secp256k1_preallocated.h · line 4 · SHA-256 1b629d577e7d70d5... · File b7283d4cc9c13bb2...

```
 3 |
 4 | #include "secp256k1.h"
 5 |
```

file_excerpt · include/secp256k1_recovery.h · line 4 · SHA-256 1b629d577e7d70d5... · File 3b2efb3ccf53e9f1...

```
 3 |
 4 | #include "secp256k1.h"
 5 |
```

file_excerpt · include/secp256k1_recovery.h · line 52 · SHA-256 995bb88bfce1fba4... · File 3b2efb3ccf53e9f1...

```
51 |     const secp256k1_context *ctx,
52 |     secp256k1_ecdsa_signature *sig,
53 |     const secp256k1_ecdsa_recoverable_signature *sigin
```

file_excerpt · include/secp256k1_recovery.h · line 84 · SHA-256 c272ad9ed8375232... · File 3b2efb3ccf53e9f1...

```
83 |  */
84 | SECP256K1_API int secp256k1_ecdsa_sign_recoverable(
85 |     const secp256k1_context *ctx,
```

## LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06410 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (3 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct. include/secp256k1_schnorrsig.h:82, src/modules/schnorrsig/main_impl.h:98 and the MuSig session at src/modules/musig/session_impl.h:583. Taproot and MuSig sit on the same hardness assumption as ECDSA: BIP-340 fixed malleability and enabled aggregation, not quantum resistance. MuSig is worth naming separately because key aggregation publishes an aggregate public key early in the protocol, so a multi-party wallet exposes a key before it is spent from.

18 reference(s) to Schnorr (BIP-340) across 3 file(s), e.g. include/secp256k1_schnorrsig.h:82. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

| | |
|---|---|
| Affected asset | source-code:Schnorr (BIP-340):transaction_signing |
| Data source | include/secp256k1_schnorrsig.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.2.0 · 2026-10-29 05:36 UTC |
| Priority score | 41.8 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include/secp256k1_schnorrsig.h:82 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · include/secp256k1_schnorrsig.h · line 82 · SHA-256 6aae3b20f8f8b503… · File 9b72c2199046d815…

```
   81 |  */
   82 | typedef struct secp256k1_schnorrsig_extraparams {
   83 |     unsigned char magic[4];
```

file_excerpt · include/secp256k1_schnorrsig.h · line 86 · SHA-256 43d5f6045ac4e6b2... · File 9b72c2199046d815...

```
   85 |     void *ndata;
   86 | } secp256k1_schnorrsig_extraparams;
   87 |
```

file_excerpt · include/secp256k1_schnorrsig.h · line 119 · SHA-256 1e555d327421285c... · File 9b72c2199046d815...

```
  118 |  */
  119 | SECP256K1_API int secp256k1_schnorrsig_sign32(
  120 |     const secp256k1_context *ctx,
```

file_excerpt · src/modules/musig/session_impl.h · line 583 · SHA-256 2e03141f0127e0e5... · File 8ddcdf7e501a4872...

```
  582 |     secp256k1_musig_nonce_process_internal(ctx, &session_i.fin_nonce_parity, fin_nonce,
&session_i.noncecoef, aggnonce_pts, agg_pk32, msg32);
  583 |     secp256k1_schnorrsig_challenge(&ctx->hash_ctx, &session_i.challenge, fin_nonce, msg32,
32, agg_pk32);
  584 |
```

file_excerpt · src/modules/schnorrsig/main_impl.h · line 98 · SHA-256 a1153ae2ef17b61b... · File 06b4bce1f64657f9...

```
   97 |  * SHA256 to SHA256("BIP0340/challenge")||SHA256("BIP0340/challenge"). */
   98 | static void secp256k1_schnorrsig_sha256_tagged(secp256k1_sha256 *sha) {
   99 |     static const uint32_t midstate[8] = {
```

file_excerpt · src/modules/schnorrsig/main_impl.h · line 106 · SHA-256 f299d1f78d362829... · File 06b4bce1f64657f9...

```
  105 |
  106 | static void secp256k1_schnorrsig_challenge(const secp256k1_hash_ctx *hash_ctx,
secp256k1_scalar* e, const unsigned char *r32, const unsigned char *msg, size_t msglen, const
unsigned char *pubkey32)
  107 | {
```

file_excerpt · src/modules/schnorrsig/main_impl.h · line 112 · SHA-256 a0a16dc204430e96... · File 06b4bce1f64657f9...

```
  111 |     /* tagged hash(r.x, pk.x, msg) */
  112 |     secp256k1_schnorrsig_sha256_tagged(&sha);
  113 |     secp256k1_sha256_write(hash_ctx, &sha, r32, 32);
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06411 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable ECDH: role not determined (1 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** The only row in this repository with a *retroactive* character, which is why it should not be read as one more signing row. include/secp256k1_ecdh.h is the key-agreement module, used by BIP-47 payment codes, silent payments and ECIES-style encryption built on top of this library. Where a shared secret derived through it protects stored or recorded data, harvest-now-decrypt-later applies: the ciphertext kept today becomes readable once a CRQC exists, and no later migration undoes that. Unlike the signature rows, that half can be fixed without a consensus change, by wrapping the agreed secret under a hybrid KEM. One row, and the first one anybody building on this library should act on.

4 reference(s) to ECDH across 1 file(s), e.g. src/bench.c:24. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

| | |
|---|---|
| Affected asset | source-code:ECDH:general |
| Data source | src/bench.c |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.2.0 · 2026-10-29 05:36 UTC |
| Priority score | 40.2 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open src/bench.c:24 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · src/bench.c · line 24 · SHA-256 6ec91ca104f465f4… · File 4108a53ceaed9109…

```
23 | #ifdef ENABLE_MODULE_ECDH
24 |     printf("    - ECDH key exchange (optional module)\n");
25 | #endif
```

file_excerpt · src/bench.c · line 58 · SHA-256 77b8586f3480b54b… · File 4108a53ceaed9109…

```
57 | #ifdef ENABLE_MODULE_ECDH
58 |     printf("    ecdh                                : ECDH key exchange algorithm\n");
59 | #endif
```

file_excerpt · src/bench.c · line 72 · SHA-256 0cfedd314441fc08… · File 4108a53ceaed9109…

```
71 |     printf("    ellswift_keygen                      : ElligatorSwift key generation\n");
72 |     printf("    ellswift_ecdh                       : ECDH on ElligatorSwift keys\n");
73 | #endif
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH; introduce an abstraction layer for crypto-agility.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06408 · CODE-QV-ASYMMETRIC

Info

## Quantum-vulnerable ECDSA: role not determined (16 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Severity dropped to informational, and this is the honest part of the result. Sixteen of the ECDSA rows and one Schnorr row could not be assigned a role because they are not uses: contrib/lax_der_parsing.{c,h} is a DER leniency helper for historical signatures, src/bench*.c and src/modules/schnorrsig/bench_impl.h are benchmarks, src/precompute_ecmult*.c generate static tables at build time, and several hits are the one-line "#include" banner at the top of include/secp256k1_*.h, which names ECDSA because every header in the family does. They are correct inventory — the primitive genuinely appears at those lines — and they are not exposure. Keeping them visible at info rather than deleting them is deliberate: a reader who checks the file list should find the same rows the tool found, and a report that silently drops the uninteresting ones cannot be reproduced. automated severity: Medium.

33 reference(s) to ECDSA across 16 file(s), e.g. contrib/lax_der_parsing.c:11. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

| | |
|---|---|
| Affected asset | source-code:ECDSA:general |
| Data source | contrib/lax_der_parsing.c |

| | |
|---|---|
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.2.0 · 2026-10-29 05:36 UTC |
| Priority score | 31.2 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

## POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

## REPRODUCTION STEPS

```
Open contrib/lax_der_parsing.c:11 and inspect the quoted lines; the evidence lists further files.
```

## EVIDENCE

file_excerpt · contrib/lax_der_parsing.c · line 11 · SHA-256 9197354c850cb53b... · File 4da2ecdc0a3b9dbb...

```
   10 |
   11 | int ecdsa_signature_parse_der_lax(const secp256k1_context* ctx, secp256k1_ecdsa_signature*
sig, const unsigned char *input, size_t inputlen) {
   12 |     size_t rpos, rlen, spos, slen;
```

file_excerpt · contrib/lax_der_parsing.c · line 19 · SHA-256 234aeaaf6b28e1d2... · File 4da2ecdc0a3b9dbb...

```
   18 |     /* Hack to initialize sig with a correctly-parsed but invalid signature. */
   19 |     secp256k1_ecdsa_signature_parse_compact(ctx, sig, tmpsig);
   20 |
```

file_excerpt · contrib/lax_der_parsing.c · line 140 · SHA-256 75ad5d0cba25ca64... · File 4da2ecdc0a3b9dbb...

```
  139 |     if (!overflow) {
  140 |         overflow = !secp256k1_ecdsa_signature_parse_compact(ctx, sig, tmpsig);
  141 |     }
```

file_excerpt · contrib/lax_der_parsing.h · line 57 · SHA-256 25af7e242d92af0d... · File 7f3e8ba009f19a5f...

```
   56 |    files (such as tests.c) without the need to set an explicit -I flag,
   57 |    which would be necessary to locate secp256k1.h. */
   58 | #ifndef SECP256K1_H
```

file_excerpt · contrib/lax_der_parsing.h · line 59 · SHA-256 4c35a81e99a7eaf5... · File 7f3e8ba009f19a5f...

```
   58 | #ifndef SECP256K1_H
   59 | #include <secp256k1.h>
   60 | #endif
```

file_excerpt · contrib/lax_der_parsing.h · line 88 · SHA-256 85efcade66268dd4... · File 7f3e8ba009f19a5f...

```
   87 |     const secp256k1_context* ctx,
   88 |     secp256k1_ecdsa_signature* sig,
   89 |     const unsigned char *input,
```

file_excerpt · contrib/lax_der_privatekey_parsing.h · line 34 · SHA-256 163cac24f6b60e79... · File 3420812d623c0f2f...

```
33 |    files (such as tests.c) without the need to set an explicit -I flag,
34 |    which would be necessary to locate secp256k1.h. */
35 | #ifndef SECP256K1_H
```

file_excerpt · contrib/lax_der_privatekey_parsing.h · line 36 · SHA-256 2741d1869779ddac… · File 3420812d623c0f2f…

```
35 | #ifndef SECP256K1_H
36 | #include <secp256k1.h>
37 | #endif
```

file_excerpt · src/bench.c · line 11 · SHA-256 c9094c1fd211f91c… · File 4108a53ceaed9109…

```
10 |
11 | #include "../include/secp256k1.h"
12 | #include "util.h"
```

file_excerpt · src/bench.c · line 100 · SHA-256 73b94c44bafc8265… · File 4108a53ceaed9109…

```
 99 |         secp256k1_pubkey pubkey;
100 |         secp256k1_ecdsa_signature sig;
101 |         data->sig[data->siglen - 1] ^= (i & 0xFF);
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06412 · CODE-QV-ASYMMETRIC

Info

## Quantum-vulnerable Schnorr (BIP-340): role not determined (1 file(s))

Long-term post-quantum risk | Confidence: Medium

**Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Severity dropped to informational, and this is the honest part of the result. Sixteen of the ECDSA rows and one Schnorr row could not be assigned a role because they are not uses: contrib/lax_der_parsing.{c,h} is a DER leniency helper for historical signatures, src/bench*.c and src/modules/schnorrsig/bench_impl.h are benchmarks, src/precompute_ecmult*.c generate static tables at build time, and several hits are the one-line "#include" banner at the top of include/secp256k1_*.h, which names ECDSA because every header in the family does. They are correct inventory — the primitive genuinely appears at those lines — and they are not

> exposure. Keeping them visible at info rather than deleting them is deliberate: a reader who checks the file list should find the same rows the tool found, and a report that silently drops the uninteresting ones cannot be reproduced. automated severity: Medium.

3 reference(s) to Schnorr (BIP-340) across 1 file(s), e.g. src/modules/schnorrsig/bench_impl.h:33. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

| | |
|---|---|
| Affected asset | source-code:Schnorr (BIP-340):general |
| Data source | src/modules/schnorrsig/bench_impl.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.2.0 · 2026-10-29 05:36 UTC |
| Priority score | 31.2 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open src/modules/schnorrsig/bench_impl.h:33 and inspect the quoted lines; the evidence lists further
files.
```

### EVIDENCE

file_excerpt · src/modules/schnorrsig/bench_impl.h · line 33 · SHA-256 fc2c5559f32fc9bd… · File 9655992c7fd21423…

```
   32 |         msg[1] = i >> 8;
   33 |         CHECK(secp256k1_schnorrsig_sign_custom(data->ctx, sig, msg, MSGLEN, data-
>keypairs[i], NULL));
   34 |     }
```

file_excerpt · src/modules/schnorrsig/bench_impl.h · line 44 · SHA-256 4f2e9b7b2529431d… · File 9655992c7fd21423…

```
   43 |         CHECK(secp256k1_xonly_pubkey_parse(data->ctx, &pk, data->pk[i]) == 1);
   44 |         CHECK(secp256k1_schnorrsig_verify(data->ctx, data->sigs[i], data->msgs[i], MSGLEN,
&pk));
   45 |     }
```

file_excerpt · src/modules/schnorrsig/bench_impl.h · line 80 · SHA-256 0ca8c9b515200145… · File 9655992c7fd21423…

```
   79 |         CHECK(secp256k1_keypair_create(data.ctx, keypair, sk));
   80 |         CHECK(secp256k1_schnorrsig_sign_custom(data.ctx, sig, msg, MSGLEN, keypair, NULL));
   81 |         CHECK(secp256k1_keypair_xonly_pub(data.ctx, &pk, NULL, keypair));
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

**RECOMMENDATION**

Classify each usage, record data lifetimes, and plan migration to a post-quantum algorithm; introduce an abstraction layer for crypto-agility.

**RETEST PROCEDURE**

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

# Evidence

| ID | Type | Data source | SHA-256 |
|---|---|---|---|
| QSI-2026-06409 | file_excerpt | include/secp256k1.h (line 75) | b6351800fde3e84aa8aa2cf6b4bf534c2ea5100a379c226f948721e4 |
| QSI-2026-06409 | file_excerpt | include/secp256k1.h (line 77) | 05e9164b839b31860fab9fa6dbbca49c52da2519809b7b247e370321 |
| QSI-2026-06409 | file_excerpt | include/secp256k1.h (line 541) | d969cc0b74d4282326ee212faae09f740473600f0b9b3ca20b935222 |
| QSI-2026-06409 | file_excerpt | include/secp256k1_ecdh.h (line 4) | 1b629d577e7d70d59383b1579315be3bf12eeca54bf16ca67903eece |
| QSI-2026-06409 | file_excerpt | include/secp256k1_ellswift.h (line 4) | 1b629d577e7d70d59383b1579315be3bf12eeca54bf16ca67903eece |
| QSI-2026-06409 | file_excerpt | include/secp256k1_extrakeys.h (line 4) | 1b629d577e7d70d59383b1579315be3bf12eeca54bf16ca67903eece |
| QSI-2026-06409 | file_excerpt | include/secp256k1_preallocated.h (line 4) | 1b629d577e7d70d59383b1579315be3bf12eeca54bf16ca67903eece |
| QSI-2026-06409 | file_excerpt | include/secp256k1_recovery.h (line 4) | 1b629d577e7d70d59383b1579315be3bf12eeca54bf16ca67903eece |
| QSI-2026-06409 | file_excerpt | include/secp256k1_recovery.h (line 52) | 995bb88bfce1fba4d66a56f3a87a98cd29136c0be9615f1b82a3199c |
| QSI-2026-06409 | file_excerpt | include/secp256k1_recovery.h (line 84) | c272ad9ed8375232acd4b7ccfcbb427af2bb07e4707e2f0c473309ff |
| QSI-2026-06410 | file_excerpt | include/secp256k1_schnorrsig.h (line 82) | 6aae3b20f8f8b503597acb97fe491246b6777b5a9660b8a41542605c |
| QSI-2026-06410 | file_excerpt | include/secp256k1_schnorrsig.h (line 86) | 43d5f6045ac4e6b23daf11fd6c10022dca16ca76afd775c3ee898ddb |
| QSI-2026-06410 | file_excerpt | include/secp256k1_schnorrsig.h (line 119) | 1e555d327421285c8a81345e58876f5f2f017571adb2f2168aafc7c4 |
| QSI-2026-06410 | file_excerpt | src/modules/musig/session_impl.h (line 583) | 2e03141f0127e0e58b8914535733b1f53be2291dc17ff0e8aaecf8ec |
| QSI-2026-06410 | file_excerpt | src/modules/schnorrsig/main_impl.h (line 98) | a1153ae2ef17b61baf8d2585aa893927fd88f650b34ce0d8166071a4 |
| QSI-2026-06410 | file_excerpt | src/modules/schnorrsig/main_impl.h (line 106) | f299d1f78d3628291910977b1ab869c1f4322bf525ee5240bf39f43f |

| | | | |
|---|---|---|---|
| QSI-2026-06410 | file_excerpt | src/modules/schnorrsig/main_impl.h (line 112) | a0a16dc204430e96c9903e6ee6ef21caae66df417cf75c5030f669ec |
| QSI-2026-06411 | file_excerpt | src/bench.c (line 24) | 6ec91ca104f465f4b861193bee0ed1f6d91c4074fb5085511ca06c7c |
| QSI-2026-06411 | file_excerpt | src/bench.c (line 58) | 77b8586f3480b54b70ea7d86972a39fdc4ee402bf83bf55c9693a2ba |
| QSI-2026-06411 | file_excerpt | src/bench.c (line 72) | 0cfedd314441fc088543e51eccc32dfe9e2e0e2e65dfaf70bf159f56 |
| QSI-2026-06408 | file_excerpt | contrib/lax_der_parsing.c (line 11) | 9197354c850cb53bbc475b348c0f97bb3f550706fcced8bf483336d6 |
| QSI-2026-06408 | file_excerpt | contrib/lax_der_parsing.c (line 19) | 234aeaaf6b28e1d2ff43854e60a3119e3c3f55229e18841e2dc76640 |
| QSI-2026-06408 | file_excerpt | contrib/lax_der_parsing.c (line 140) | 75ad5d0cba25ca64adb6d971da937879704b4abc8845c17dc1da9fa1 |
| QSI-2026-06408 | file_excerpt | contrib/lax_der_parsing.h (line 57) | 25af7e242d92af0d3c46ae8d8a48030e247c4bfdeba65068030e64eb |
| QSI-2026-06408 | file_excerpt | contrib/lax_der_parsing.h (line 59) | 4c35a81e99a7eaf5ea7315346f3b8cacb0832f7c4a3b1a3d003ff0bc |
| QSI-2026-06408 | file_excerpt | contrib/lax_der_parsing.h (line 88) | 85efcade66268dd4d0b42e8f31b634ea4cd93d9d55cdc2e036715108 |
| QSI-2026-06408 | file_excerpt | contrib/lax_der_privatekey_parsing.h (line 34) | 163cac24f6b60e798317e2383fb603da718e728035262dc9759d7cf3 |
| QSI-2026-06408 | file_excerpt | contrib/lax_der_privatekey_parsing.h (line 36) | 2741d1869779ddac4f151ac1740c338ba299b2bed7b98a38e6042acb |
| QSI-2026-06408 | file_excerpt | src/bench.c (line 11) | c9094c1fd211f91c6bc608456e54a86301912edf8adab7b826ff6ed1 |
| QSI-2026-06408 | file_excerpt | src/bench.c (line 100) | 73b94c44bafc8265f5a60c3a8022ff9334fa9b8b57c14d4ddfe01e08 |
| QSI-2026-06412 | file_excerpt | src/modules/schnorrsig/bench_impl.h (line 33) | fc2c5559f32fc9bd9a85efef59a46eaf722b1b8e2745b837696cdf6a |
| QSI-2026-06412 | file_excerpt | src/modules/schnorrsig/bench_impl.h (line 44) | 4f2e9b7b2529431d8190d606bbe0ce49659d6e5a1e0410625d931eac |
| QSI-2026-06412 | file_excerpt | src/modules/schnorrsig/bench_impl.h (line 80) | 0ca8c9b51520014576b4c622e33014e8b93cc7c4bd0bc826e9cd4563 |

# Post-quantum assessment

| | Key establishment (x = how long data must stay confidential; harvest now, decrypt later) | Signatures (x = how long signatures/ keys must stay trusted) |
|---|---|---|
| Data/key lifetime x | 10 | 10 |
| Migration time y | 5 | 5 |
| Assumed years to CRQC z | 8 (early) | 8 |
| x + y − z | 7 → migration would finish too late under this scenario | 7 → migration would finish too late under this scenario |

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

## Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography

**Next step:** Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

## What the quantum-vulnerable algorithms do

| Role | Algorithms (files) | ID | First step |
|---|---|---|---|
| **transaction and message signing** | ECDSA (10), Schnorr (BIP-340) (3) | QSI-2026-06409, QSI-2026-06410 | Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them |
| **role not determined** | ECDSA (16), ECDH (1), Schnorr (BIP-340) (1) | QSI-2026-06408, QSI-2026-06411, QSI-2026-06412 | Classify each usage, record data lifetimes, and plan migration to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH; introduce an abstraction layer for crypto-agility |

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

| Quantum status | Count |
|---|---|
| vulnerable (Shor) | 38 |

**Detailed findings:** QSI-2026-06409, QSI-2026-06410, QSI-2026-06411, QSI-2026-06408, QSI-2026-06412

## PQC readiness questionnaire

No readiness questionnaire was completed for this project.

## Quantum algorithms relevant to this inventory

| Quantum algorithm | Effect | Inventoried algorithms (rows) | What it means for this audit |
|---|---|---|---|
| Shor's algorithm — elliptic-curve discrete logarithms<br>Q1: quantum computation offline, classical access to the victim | breaks | ECDH (2), ECDSA (32), Schnorr (BIP-340) (4) | Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime. |

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

# Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

# Remediation plan

QSI-2026-06409 · P3 · 90 days

short-term risk reduction

### Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-06410 · P3 · 90 days

short-term risk reduction

### Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-06411 · P3 · 90 days

short-term risk reduction

### Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |

| | |
|---|---|
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-06408 · P3 · 90 days

short-term risk reduction

### Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-06412 · P3 · 90 days

short-term risk reduction

### Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-06409 · P4 · 90 days

root-cause fix

### Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | |

| | |
|---|---|
| | Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use. |

QSI-2026-06410 · P4 · 90 days root-cause fix

### Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows or a hybrid scheme in use. |

QSI-2026-06411 · P4 · 90 days root-cause fix

### Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH or a hybrid scheme in use. |

QSI-2026-06408 · P4 · 90 days root-cause fix

### Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |

| | |
|---|---|
| Closure criteria | Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use. |

QSI-2026-06412 · P4 · 90 days

root-cause fix

### Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows or a hybrid scheme in use. |

# Priorities for 30, 60 and 90 days

## 30 days

| Priority | ID | Action | Type |
|---|---|---|---|
| None | | | |

## 60 days

| Priority | ID | Action | Type |
|---|---|---|---|
| None | | | |

## 90 days

| Priority | ID | Action | Type |
|---|---|---|---|
| P3 | QSI-2026-06409 | Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-06410 | Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-06411 | Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-06408 | Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-06412 | Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P4 | QSI-2026-06409 | Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-06410 | Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-06411 | Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-06408 | Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |

| | | | |
|---|---|---|---|
| P4 | QSI-2026-06412 | Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |

# Verification after remediation

| ID | Retest procedure |
|---|---|
| QSI-2026-06409 | Re-run the inventory after refactoring. |
| QSI-2026-06410 | Re-run the inventory after refactoring. |
| QSI-2026-06411 | Re-run the inventory after refactoring. |
| QSI-2026-06408 | Re-run the inventory after refactoring. |
| QSI-2026-06412 | Re-run the inventory after refactoring. |

# Tools used

| Tool | Version |
|---|---|
| blockchain | 1.0.0 |
| certificates | 1.1.0 |
| chain_map | 1.0.0 |
| configs | 1.1.0 |
| dependencies | 1.2.0 |
| inventory_document | 1.1.0 |
| solidity | 1.2.0 |
| source_code | 1.3.0 |
| tls_code | 1.1.0 |
| tls_live | 1.0.0 |
| qsi-engine | 1.2.0 |
| OSV.dev API | v1 |

# Sources and normative documents

Reviewed: 2026-10-13


1. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
2. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
3. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
4. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
5. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
6. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
7. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
8. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
9. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)
10. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)
11. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/ipd (initial public draft (November 2024); proposes deprecating quantum-vulnerable public-key algorithms after 2030 and disallowing them after 2035 — dates may change in the final version)
12. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)
13. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)

# Appendices

## A. Cryptographic inventory

| Algorithm | Parameter | Quantum status | Classical status | Count | Locations | Examples |
|---|---|---|---|---|---|---|
| ECDH | | vulnerable (Shor) | acceptable | 2 | 2 | include/secp256k1_ecdh.h; src/bench.c:24 |
| ECDSA | | vulnerable (Shor) | acceptable | 9 | 9 | contrib/lax_der_parsing.c:11; contrib/lax_der_parsing.h:88; include/secp256k1.h:75 |
| ECDSA | secp256k1 | vulnerable (Shor) | acceptable | 23 | 23 | contrib/lax_der_parsing.h:57; contrib/lax_der_privatekey_parsing.h:34; include/secp256k1_ecdh.h:4 |
| Schnorr (BIP-340) | | vulnerable (Shor) | acceptable | 4 | 4 | include/secp256k1_schnorrsig.h:82; src/modules/musig/session_impl.h:583; src/modules/schnorrsig/bench_impl.h:33 |

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

## B. JSON

qsi.report/1 · SHA-256 169d88e86f2f6d7d0b7b36eaadbd0524a11b958affb0f07abddec70ff1d2820a

## C. Input files

| File | Size | SHA-256 | Reda |
|---|---|---|---|
| contrib/lax_der_parsing.c | 3595 | 4da2ecdc0a3b9dbb9029d188bb0243764a3c2f2324613a688f1ac4d3fc878851 | 0 |
| contrib/lax_der_parsing.h | 4230 | 7f3e8ba009f19a5f2759c9065af02cc8e62351100d69f8ae7e762fa4b65ea675 | 0 |
| contrib/lax_der_privatekey_parsing.c | 5112 | 48352c75520eecd141936c0011601925510cdf60afd3e46e79f0a0547cbc78da | 0 |
| contrib/lax_der_privatekey_parsing.h | 4016 | 3420812d623c0f2f86debe0535f98eab9b929036bb2fee2de8e3b1adc3e1a0b9 | 0 |
| include/secp256k1.h | 44715 | f9af5be29b0fbe1a2b65d4131e77832791d70d995472603c124b90fb5b427538 | 0 |
| include/secp256k1_ecdh.h | 2560 | ad314d14cb79a79d4113862f21e0deccf37962db7811c801dab2899f91dd8222 | 0 |
| include/secp256k1_ellswift.h | 9148 | 620db468b1c13968946e9f404ceb59a3f11c895603bf1dcd3399567d7bf10e99 | 0 |
| include/secp256k1_extrakeys.h | 10870 | 7c3468c9fd192ad152c5cf042cb44938961d752fcf19966123ce6a5e3fd31637 | 0 |
| include/secp256k1_musig.h | 28115 | b7e7772dfc78585dac1d7c2921a44dc62b3aec92df50818fdf985eaeb88139ae | 0 |
| include/secp256k1_preallocated.h | 5966 | b7283d4cc9c13bb2572dc92e69cea76f3270403baf5748ea4d91a1064b951b85 | 0 |
| include/secp256k1_recovery.h | 5225 | 3b2efb3ccf53e9f1bce93a4afae31946585958d5cba3b38fdfb1eea4c6ce2622 | 0 |
| include/secp256k1_schnorrsig.h | 7698 | 9b72c2199046d815df533d61a0287c1f4ca73cdef2195666c0c57d284e0c84cc | 0 |

| | | | |
|---|---|---|---|
| include/secp256k1_silentpayments.h | 20601 | 70b073de4d41c4ce33b723bd10ed46453d23ea5c41b0a4a034c0d0988a56da8c | 0 |
| src/assumptions.h | 4156 | e2530719f50131f21c7d5fa455b71c108050b914a0537942a470ca1f925256e8 | 0 |
| src/bench.c | 11727 | 4108a53ceaed91090a8ec53470f7310526a5234a4c8258b416b514b4c4535ef5 | 0 |
| src/bench.h | 4777 | 575fc51f87564f46856959f88753b37b3c4c1f09c772e7f43a5d59c68f24e9c9 | 0 |
| src/bench_ecmult.c | 15662 | ad3854823478b46a2b541bb85634e3b122800f4082dd22af38a3bd6e7275adde | 0 |
| src/bench_internal.c | 18493 | e9df9ba3bb8dff381dbcb06d578693b0f3324bef581cebb2df3164f65e0ffbc4 | 0 |
| src/checkmem.h | 5355 | f367089bffa53b8998047de2f718f75d627dad549a1e5dc2a9ce9ab9f581ffe8 | 0 |
| src/ecdsa.h | 1175 | 3795a37c5ee8c53aea0dc69adabf8ac29ec2b1ab10e613ebfd2816cdc8285d41 | 0 |
| src/ecdsa_impl.h | 10926 | 390402c9b04f0807df27e0f82afe235c47fdb5ccd2d6a6483c29767cbc882bb3 | 0 |
| src/eckey.h | 945 | 3ca0eac1cadf9ef6ab1d5c2d0bdb27538577bda3fe8edde68f0e8d94351e9146 | 0 |
| src/eckey_impl.h | 1652 | 44cc9d08f952ea973cc7c23834f2c36b2198e74008dc21d9d6734bcc00193b52 | 0 |
| src/ecmult.h | 3177 | 4b6b14f62fc88e83cb6cf1d8f59f9ee0ae62218f91d579a76614553ac46cedc5 | 0 |
| src/ecmult_compute_table.h | 1073 | 5a8ce69b18d777c3ad608ef173fd6a47c75b9c37bc6801f19ca4c44eb4b9f8f6 | 0 |
| src/ecmult_compute_table_impl.h | 1916 | 6082339f616cd5a628300df24e66353c1374a117e24ddd474fe6151dbc620d60 | 0 |
| src/ecmult_const.h | 1468 | d0ef8a4997ac4079f0c0bcf824948d111a0b04e58fd2f95975e346df2592b01b | 0 |
| src/ecmult_const_impl.h | 20165 | b72a245f660375c6eb55f54e77ef6c8e11781429793039caf5090bd4a806127a | 0 |
| src/ecmult_gen.h | 5639 | b59f002fe1f686db7c852d637040f573d8b36dc76905ad7d293c1a2f8f43e645 | 0 |
| src/ecmult_gen_compute_table.h | 672 | 51eb54b0cb7b49b25cf751efb2799322d73c97bc9bdd2a263feff2d2d6eeff9f | 0 |
| src/ecmult_gen_compute_table_impl.h | 4705 | 9ae524743bb5871bff7e41e48b61218336b1919769132aafa944283ef1bb88f2 | 0 |
| src/ecmult_gen_impl.h | 17232 | c03ba593db67e430f000d7d7b0305be9c020cdb91a407b9dc4dfbe40ce674332 | 0 |
| src/ecmult_impl.h | 33871 | 4fba54d427015fe9e552732c281e4b33083b8340a5383875e1f03af2b2bf234e | 0 |
| src/field.h | 15462 | 327585fdf056f1c87fd3ae68181b8eb06a36a5f1f6ce839082ded5fce861437f | 0 |
| src/field_10x26.h | 2338 | ba9a43b62e7e70e6e18af2787bd01dfc3a57fea527cf8ad0333194211ac7d7de | 0 |
| src/field_10x26_impl.h | 51376 | eee15d91fa24be60104c2e24e9744340a1b55db40986fcc8e5ea8eaa27e9ea7a | 0 |
| src/field_5x52.h | 2380 | cbaa91c253ba84d99efcd334c7a39906c164d5edf3cde81200a44a606205cdce | 0 |
| src/field_5x52_impl.h | 18483 | 87acd72100f54b8087a8ecc77ac726689ceef0bf20d0f5c62b0b2797881572e4 | 0 |
| src/field_5x52_int128_impl.h | 11353 | 9f3071c7da547b94e0e6eabe0dbedc42535ea7c470f1dc891d0a24aa5d19dcf7 | 0 |
| src/field_impl.h | 14082 | 8ffb63295d21910c3db8d8c1f3154620b25bb8df963d0f2234442e7bf96c19d5 | 0 |
| src/group.h | 13556 | aa3a2b432b218f0ae87b727847fa1c1a5801fbb9e7504ff06bea86f59f5cd707 | 0 |
| src/group_impl.h | 44387 | 8ec8e7eb698346114cdeef81a769c55ba8e54764d7f183a56e887c0125e1eb71 | 0 |
| src/hash.h | 2660 | 4a30c765a9bd49b57bba6a0879fdea66f915f606e6f9f4c92abf580668d50999 | 0 |
| src/hash_impl.h | 18679 | e7d20d64807cda921dbb816fdc80085287272a718cc5f97c15720449317196d2 | 0 |

| | | | |
|---|---|---|---|
| src/hsort.h | 1685 | 220cb669835d34c252598ecff860c93edda337a9f8c2fc07b2b6cead7d48b284 | 0 |
| src/hsort_impl.h | 4798 | 2861489c01b927b27d99efe173799a812434748b4addd30d102187ce4f3b66a9 | 0 |
| src/int128.h | 3622 | 89ac5d91d64f857038d063dd70eec4b29b4f32aa218c790fe049137c347186cb | 0 |
| src/int128_impl.h | 376 | 5683c02dfc495639da259d80624711c1c449bbaee6186729e399e713a28b4719 | 0 |
| src/int128_native.h | 581 | b4c52c7ff091083f5eca9879b98641c7a38356a3c5321cbcc22ae1573e7711e1 | 0 |
| src/int128_native_impl.h | 2881 | ecaafefadbde3db034d4f44678096833d2d113fd21b4ff7044095b6153ec03a8 | 0 |
| src/int128_struct.h | 229 | 0733b3a65719a1e9a8fb3564c71a38f2f31ca3522f40087d536dba90361b66e7 | 0 |
| src/int128_struct_impl.h | 7884 | 0b4e2200327f5357078ba43b0e0514c58c2b1eb217b87ebc14d95efb913b5be2 | 0 |
| src/modinv32.h | 1865 | 774887ccd462b145434d1d6b9ae9455aa3618e791544a2b61267e18931d5010d | 0 |
| src/modinv32_impl.h | 33492 | befe02af7099a871677c7b05a81302dff5a4dc1d59f19b20bcb58cc28e096112 | 0 |
| src/modinv64.h | 1969 | b71cf177d768beac28fcb4f0ef545aafaad16ad0f275c70ee4c60b5f7549ac41 | 0 |
| src/modinv64_impl.h | 37653 | 1254dc7ea941fc4d4ef0a85eed80db58d9e2647a8b8d19238d6afc048d63764d | 0 |
| src/modules/ecdh/bench_impl.h | 1761 | 3929ead352b3b6425f0fdf94a1b2268d124cc0176929ae6ea3b4fa82abac0bbf | 0 |
| src/modules/ecdh/main_impl.h | 2829 | f28e34d72a3eeb1c726a8a8b390984eeef17089a5316153b71e234d9cf6c286f | 0 |
| src/modules/ellswift/bench_impl.h | 4399 | 8973d64bda27eb72a4807aae665a1a2932d624370f9d40f8833f7c8f03b05564 | 0 |
| src/modules/ellswift/main_impl.h | 27492 | 5cafe952c236510ba2352ed59f6cca808037bed210bd76cbf3610aa7e2d3aaa3 | 0 |
| src/modules/extrakeys/main_impl.h | 10195 | 6c002046bda20ffb8767cac1c6800dd3b8e9be9e648772a1498539159a4b4937 | 0 |
| src/modules/musig/keyagg.h | 1324 | 5349443b84525a85434ff8f92a1e7542c0982b1b783a682613026203de4148dd | 0 |
| src/modules/musig/keyagg_impl.h | 11110 | 8a00bc5296e360cce8a94ffe889e61fbfb87448fdc4c25220218c35aeab11cf9 | 0 |
| src/modules/musig/main_impl.h | 426 | 9ec7e6082aa496353466769bb9f7705c32caf8d060b0c92815cb37f6c4265654 | 0 |
| src/modules/musig/session.h | 861 | c256c3742a68849520569845640f152a22a4a8c21a96040cf0c1054516983980 | 0 |
| src/modules/musig/session_impl.h | 30011 | 8ddcdf7e501a487213289db68f45e4183bd9cba6901c10c329b82efaecc24cf3 | 0 |
| src/modules/musig/vectors.h | 37797 | c7cd9dc3804d864d17e43c5b00335a405a3bc4a30861838ac4599a213231e132 | 0 |
| src/modules/recovery/bench_impl.h | 2320 | 20d13192a4aabaa52ac4c2a2da772d8aafef00773051836b25faa2816226b386 | 0 |
| src/modules/recovery/main_impl.h | 6135 | fbec2358a560b669d85720108b0b1a8f653c91531a88ded0a1da339445266cc9 | 0 |
| src/modules/schnorrsig/bench_impl.h | 3795 | 9655992c7fd214231623716a738a1928e24a86b4bcdd909f331712df800a1abc | 0 |
| src/modules/schnorrsig/main_impl.h | 10555 | 06b4bce1f64657f9a21b78de77e8215e2d0b5f9da128fe4fa5f8dfe27f5dfe8e | 0 |
| src/modules/silentpayments/bench_impl.h | 10607 | 688e6638662ff8b0d60810c0d4e2d901f7e52d15e1e703bd566ab8577d4f3c71 | 0 |

| | | | |
|---|---|---|---|
| src/modules/silentpayments/main_impl.h | 37953 | 5a6a8a55d141bd76689653d530cc4aa25e297d6d309985aab0b02e67d3a26cd7 | 0 |
| src/modules/silentpayments/vectors.h | 539457 | 8d88aead1f2f359aca31ac8c803001c55b1231be187839cafbe9c6959cedbbcc | 0 |
| src/precompute_ecmult.c | 4017 | 8085ea86390eab2afeeec8b4f952036492fb81606dcd7bb038f940924b1b8400 | 0 |
| src/precompute_ecmult_gen.c | 3864 | 23d1dadb40296d2752d024678e405b18d5c904fbd61d74598df3c8005ba64d2c | 0 |
| src/precomputed_ecmult.c | 2409168 | 1fba19c482de09f566ed9e7bf6a526e4edb59a1cf38218e3ab413000a0e0deda | 0 |
| src/precomputed_ecmult.h | 1585 | d792b8a49759deff5f1e4092558fa9ae11d24e02695e44bee63a8f0ba8e18503 | 0 |
| src/precomputed_ecmult_gen.c | 259715 | c0a7af0884b7b07962be67e9da674e0d75a8d412d116153b8eae2a9e5527d401 | 0 |
| src/precomputed_ecmult_gen.h | 973 | 8f514bed441bccf7e34ff05ff49d3a44e9395519a8fdcf176e6aeab7b5dbd5fc | 0 |
| src/scalar.h | 5463 | 4e9622fb9b90e971ee40f1f5b7598fc5067870512db1ace4241333d1329e856b | 0 |
| src/scalar_4x64.h | 801 | 42aed583b28bda7982c54a6f63e50c0ae5f9c27d5f66d92ec6bf6c90e765735a | 0 |
| src/scalar_4x64_impl.h | 33058 | 92bc1023ee0e53cbd955a779b2f70d8b29bdc51ca02b61f23dda7054455264de | 0 |
| src/scalar_8x32.h | 725 | 0614cea113aea88a47ef3b2ba8167722e850d7b2a696a4f16b03ec9a320ef1d6 | 0 |
| src/scalar_8x32_impl.h | 29203 | 19ad58cb2a3699c979ee2baf072586302ff2b3a7cfe59c7bd91c5c01120e8727 | 0 |
| src/scalar_impl.h | 12113 | 6ba654bd4edc0349d1d7f1dfc25ba2d73dd9f2ecdca8078e384bf7b7837d663f | 0 |
| src/scalar_low.h | 1135 | 3cab20db10224ff85ee7c6d975d2968c1370ed8e96ea26b94650f6e2bac31ac4 | 0 |
| src/scalar_low_impl.h | 6212 | 188ea86c20ada218798c558fed63f0ed6267e3cf5ee7e79be2db07cc8584c064 | 0 |
| src/scratch.h | 2224 | 4af5e1ff665c0418bcc5b1c2de954e7b377f9bdfdd4309e0296938918fac11ef | 0 |
| src/scratch_impl.h | 3837 | 6a9e8b22fc4b17a0d90e71db7e32584c3aafefc1a4a18ec037a859c6c0ee19b0 | 0 |
| src/secp256k1.c | 30428 | 07513565cdd1af4d528228626350dbd1eecfc6e786c242f63f9c91f4c28b39c5 | 0 |
| src/selftest.h | 1659 | 2eda8afb246812b36e9761977c93ed75a95e1ccfbdab1c31ffba4986b937335e | 0 |
| src/testrand.h | 1813 | 716ca2566258ae09b04e27df76d2c2c5d3c1614a891461fdf0e5aa126d0cf819 | 0 |
| src/testrand_impl.h | 5340 | 39c875e0b2cd9b93cdd7e71db009e8486d55379d4e1f2f4d2eb4cd882cf54e2b | 0 |
| src/testutil.h | 4945 | 5a780fc16fcd8549ea4774d15beb2d36c6ad9a2ca7180dd95f84199a2863d1ff | 0 |
| src/util.h | 16431 | 93ebecb8e8c43aab9bc6f0a2b3ed32027a14f7eb81e46194a3b8a3ea92049c9f | 0 |
| src/util_local_visibility.h | 402 | d47d4a58f40157ab2a298151f366c5ebec8f7a1e7aaba88e36b530b1c01584ca | 0 |
| tools/symbol-check.py | 2416 | 1706069df728c1a2ac1d2578897076713c16eed73315be6c83ad9690007bca70 | 0 |
| tools/wycheproof_utils.py | 427 | 9f2c2368953eeb3c9b7c3b31a807cde76b3106df93fbb975993638d5eb1a1e8a | 0 |